



RAG 설계서

담당자	민희 [INT1]한민희
진행 상태	완료
프로젝트	주제 선정/기획
우선순위	높음
태그	

관리규정 RAG 설계서

1. 문서 개요

1-1. 목적

본 문서는 아파트 관리규정에 대한 **검색 증강 생성(RAG, Retrieval-Augmented Generation) 시스템**의 구조와 데이터 처리 방식을 정의한다.

본 시스템은 여러 아파트의 관리규정 문서를 수집·저장하고, 사용자의 자연어 질문에 적합한 규정을 검색한 후 Amazon Bedrock을 활용하여 **근거 조항과 함께 답변을 생성**하는 것을 목표로 한다.

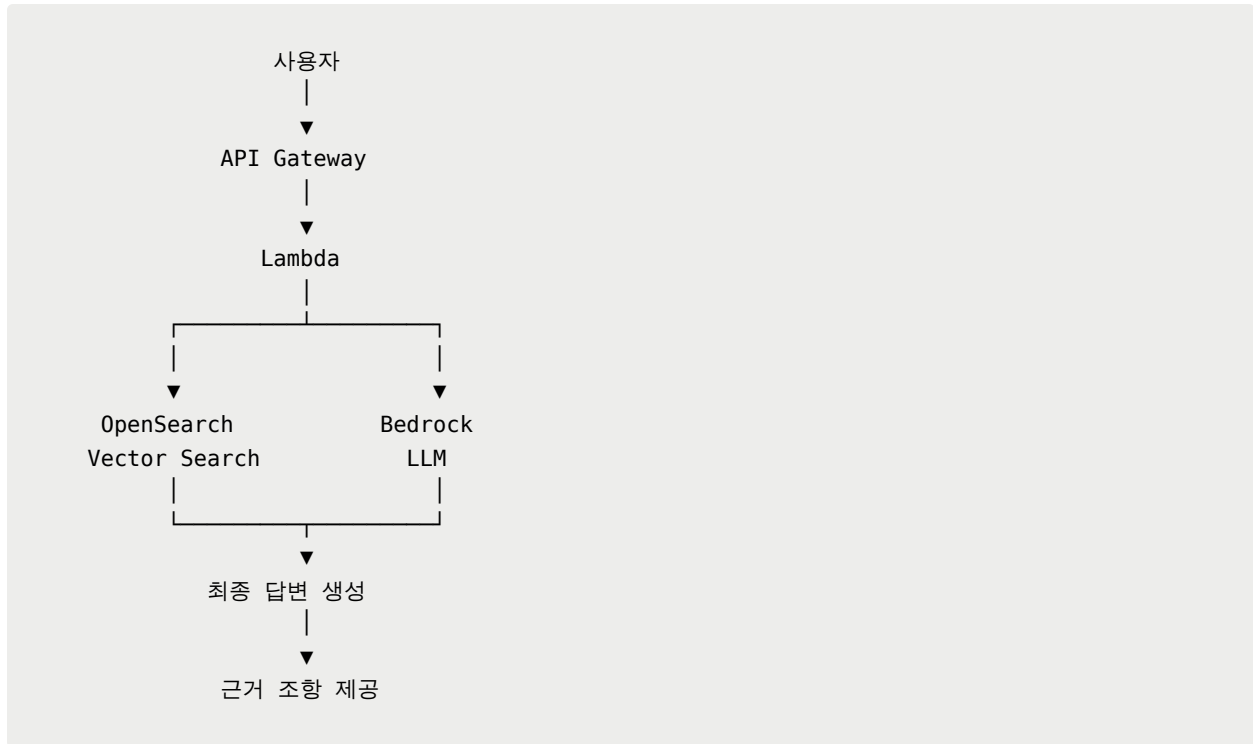
특히 다음 두 가지 검색 시나리오를 지원한다.

1. 아파트를 지정하지 않은 질문 → **사용자의 소속 아파트 규정 검색**
2. 특정 아파트를 지정한 질문 → **해당 아파트의 공개 규정 검색**

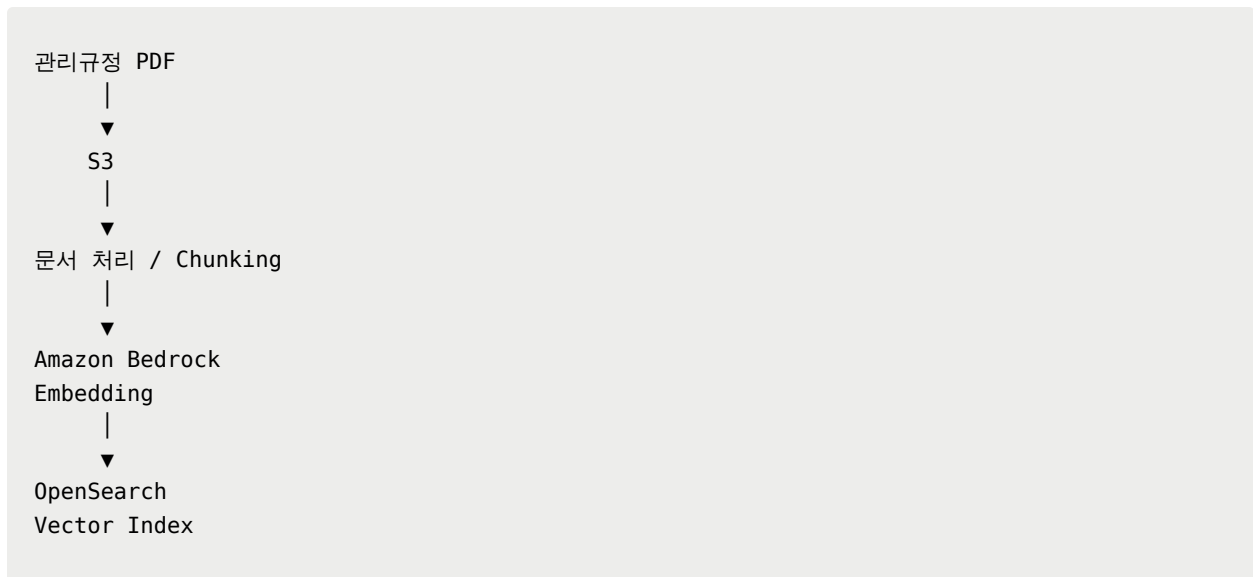
또한 여러 아파트가 질문에 포함된 경우 각 아파트의 공개 규정을 검색하여 비교할 수 있도록 설계한다.

2. RAG 시스템 구조

2-1. 전체 구조



문서 등록 및 갱신은 별도의 데이터 처리 파이프라인으로 구성한다.



3. 사용 기술

구성 요소	기술	역할
문서 저장소	Amazon S3	관리규정 PDF 원본 저장
API	Amazon API Gateway	RAG 질의 API 제공
처리 서버	AWS Lambda	질의 분석 및 RAG 파이프라인 실행
Vector DB	Amazon OpenSearch	문서 Chunk 및 Embedding 검색
Embedding	Amazon Bedrock	문서/질문 Vector 생성
LLM	Amazon Bedrock	검색 결과 기반 답변 생성

4. 데이터 구조

4-1. 관리규정 문서 구조

관리규정은 아파트 단위로 식별할 수 있어야 한다.

```
S3
├── apartment-A/
│   ├── management-rules.pdf
│   ├── parking-rules.pdf
│   └── facility-rules.pdf
├── apartment-B/
│   ├── management-rules.pdf
│   └── parking-rules.pdf
└── apartment-C/
    ├── management-rules.pdf
    └── noise-rules.pdf
```

각 문서는 최소한 다음 메타데이터를 가진다.

메타데이터	설명
<code>apartment_id</code>	아파트 식별자
<code>apartment_name</code>	아파트명

메타데이터	설명
<code>document_id</code>	문서 식별자
<code>document_name</code>	규정 문서명
<code>document_type</code>	관리규약 / 주차 / 시설 / 층간소음 등
<code>is_public</code>	외부 아파트 사용자에게 공개 가능한지 여부
<code>version</code>	문서 버전
<code>page</code>	원본 PDF 페이지
<code>updated_at</code>	문서 수정일

5. 문서 수집 및 저장

5-1. 문서 등록

관리자는 자신이 관리하는 아파트의 관리규정 PDF를 등록한다.

```

관리자
↓
관리규정 PDF 업로드
↓
S3
↓
문서 처리

```

등록 가능한 문서의 예시는 다음과 같다.

- 아파트 관리규약
- 주차관리규정
- 시설이용규정
- 층간소음규정
- 기타 관리규정

5-2. 문서 버전 관리

관리규정이 변경되면 기존 문서를 무조건 삭제하는 것이 아니라 **문서 버전을 구분**한다.

예:

주차관리규정

v1

2026.01.01

v2

2026.07.01

검색 대상은 기본적으로 최신 활성 문서로 제한한다.

이를 통해 관리규정이 변경되었을 때 이전 규정이 검색되는 문제를 방지한다.

6. 문서 전처리

PDF 원문을 그대로 Vector DB에 저장하지 않고 검색 가능한 단위로 분할한다.

PDF

↓

Text Extraction

↓

문서 구조 분석

↓

Chunking

↓

Metadata 추가

↓

Embedding

↓

OpenSearch 저장

6-1. Chunking 기준

가능하면 단순한 글자 수 기준보다는 **관리규정의 조항 구조**를 유지한다.

예:

제15조 주차관리

└─ 제1항

└ 제2항
└ 제3항

하나의 Chunk에 가능한 한 다음 정보가 함께 포함되도록 한다.

- 규정명
- 장/절
- 조
- 항
- 호
- 본문

이를 통해 검색 결과를 답변의 근거로 활용하기 쉽게 한다.

7. Embedding

각 Chunk를 Amazon Bedrock의 Embedding 모델을 이용하여 Vector로 변환한다.

```
관리규정 Chunk
  ↓
Bedrock Embedding
  ↓
Vector
  ↓
OpenSearch
```

OpenSearch에는 Vector뿐만 아니라 검색 및 답변에 필요한 원문 Metadata도 함께 저장한다.

8. OpenSearch Vector Index 설계

각 Vector Document는 다음과 같은 구조를 가진다.

```
{
  "apartment_id": "APT001",
```

```

"apartment_name": "00아파트",
"document_id": "DOC001",
"document_type": "PARKING",
"document_name": "주차관리규정",
"version": "2",
"is_public": true,
"chapter": "제3장 주차관리",
"article": "제15조",
"paragraph": "제2항",
"page": 8,
"content": "방문 차량은 ...",
"embedding": [...]
}

```

핵심은 `apartment_id` 와 `is_public` 을 검색 Metadata로 사용하는 것이다.

9. 질문 처리

9-1. 기본 질문

사용자가 다음과 같이 질문한다고 가정한다.

└ "주차 등록은 어떻게 해?"

질문에 아파트가 명시되어 있지 않으므로 사용자의 소속 아파트를 기준으로 검색한다.

```

사용자 질문
↓
아파트 정보 확인
↓
사용자 소속 아파트 = A아파트
↓
A아파트 규정 검색
↓
관련 Chunk 검색
↓
Bedrock

```

↓
답변 생성

10. 특정 아파트 검색

사용자가 특정 아파트를 질문에 명시할 수 있다.

|"B아파트 주차 규정은 어떻게 돼?"

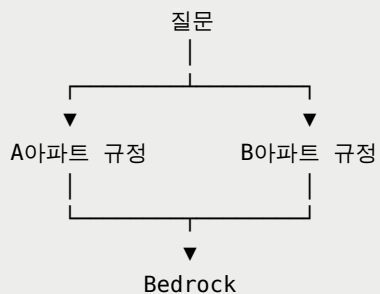
이 경우 질문에서 B아파트를 식별하고 B아파트의 공개된 규정만 검색한다.

질문
↓
아파트명 추출
↓
B아파트
↓
is_public = true
↓
B아파트 규정 검색
↓
Bedrock
↓
답변

11. 다중 아파트 비교

여러 아파트가 질문에 포함되는 경우 각각의 공개 규정을 검색한다.

|"A아파트와 B아파트의 주차 규정 차이를 알려줘."



↓
비교 결과 생성

답변 예시

항목	A아파트	B아파트
방문차량 등록	필요	필요
등록 가능 시간	24시간	07~22시
무료 주차	2시간	3시간

각 비교 내용에는 해당 아파트의 근거 조항을 함께 제공한다.

12. 검색 범위 제어

RAG 시스템에서는 검색 전에 검색 범위를 결정하는 과정이 중요하다.

Case 1. 아파트 미지정

```
"주차비는 얼마야?"  
↓  
사용자 소속 아파트 확인  
↓  
apartment_id = 사용자 아파트  
↓  
검색
```

Case 2. 특정 아파트

```
"00아파트 주차비는?"  
↓  
00아파트 식별  
↓  
apartment_id = 00아파트  
is_public = true  
↓  
검색
```

Case 3. 여러 아파트

"A와 B 아파트의 주차 규정은?"

↓

A + B 식별

↓

각 아파트별 검색

↓

비교

13. 일반 서비스 데이터와 RAG 데이터의 분리

RAG 시스템에서 가장 중요한 보안 원칙은 일반 아파트 데이터와 관리규정 데이터를 분리하는 것이다.

일반 서비스

PostgreSQL

A아파트

- ├ 관리비
- ├ 공지사항
- ├ 민원
- └ 긴급알림

사용자는 본인 아파트의 데이터만 조회한다.

RAG

OpenSearch

A아파트

- ├ 공개 규정
- └ 비공개 규정

B아파트

- ├ 공개 규정
- └ 비공개 규정

RAG에서는 특정 아파트를 요청하더라도 `is_public = true` 인 문서만 검색한다.

따라서

|"B아파트 관리비가 얼마야?"

와 같은 질문은 RAG에서 답변할 수 있는 대상이 아니다.

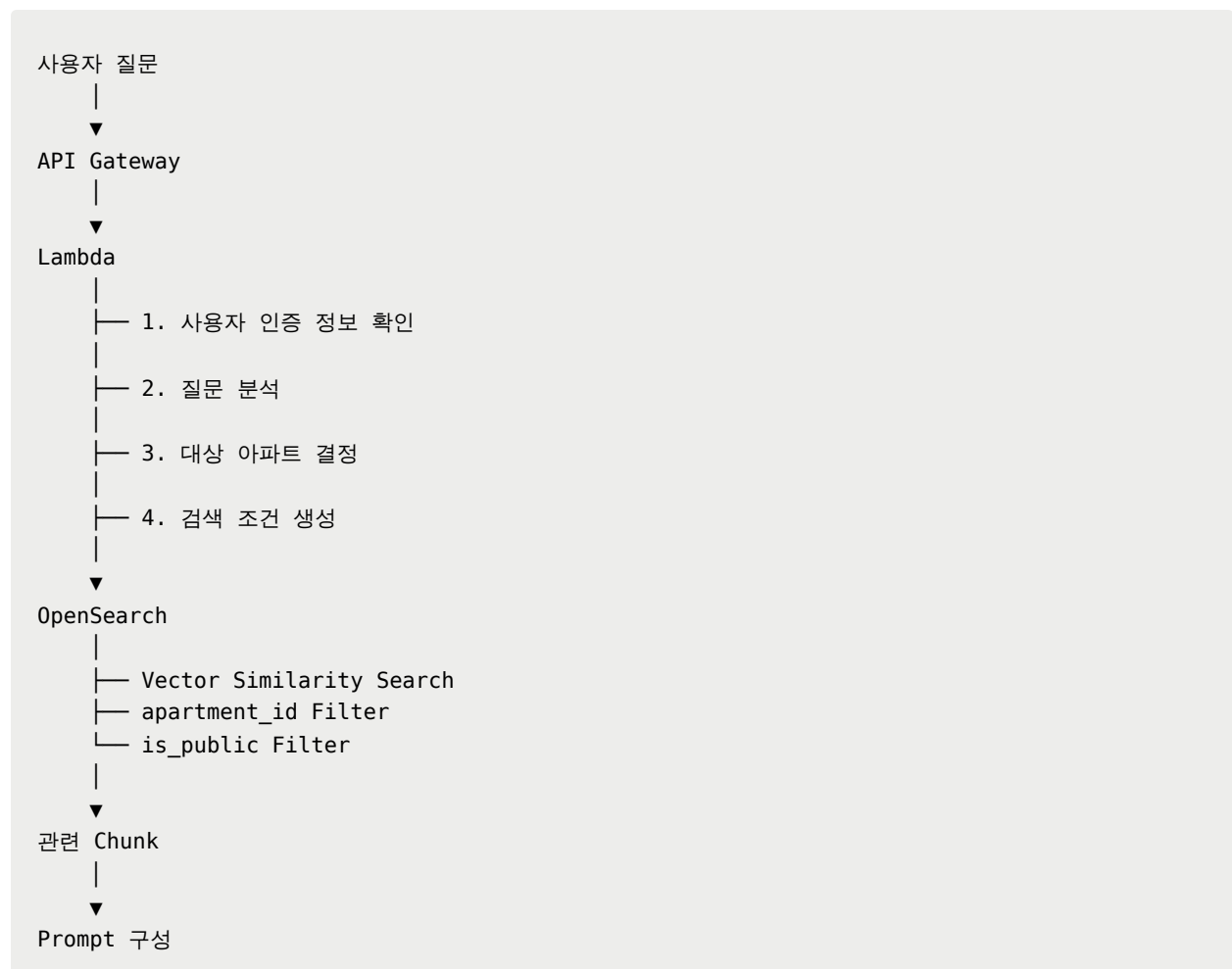
반면

|"B아파트 주차 규정이 어떻게 돼?"

는 B아파트의 공개 주차관리규정을 검색하여 답변할 수 있다.

14. RAG 질의 처리 Pipeline

전체 질의 처리는 다음과 같이 구성한다.





15. Prompt 구성

검색된 문서를 Context로 제공하고, LLM이 Context를 기반으로 답변하도록 구성한다.

```
System Prompt
+
사용자 질문
+
검색된 규정 Context
+
아파트 정보
↓
Amazon Bedrock
↓
최종 답변
```

Prompt에는 다음과 같은 원칙을 적용한다.

답변 원칙

1. 검색된 관리규정을 우선하여 답변한다.
2. 근거가 없는 내용을 임의로 생성하지 않는다.
3. 답변에 근거 조항을 포함한다.
4. 규정에서 확인할 수 없는 경우 이를 명확하게 표시한다.
5. 여러 아파트가 포함된 경우 아파트별 규정을 구분하여 답변한다.

16. 답변 형태

RAG 답변은 단순한 자연어 답변이 아니라 **답변 + 근거** 구조로 제공한다.

[답변]

방문 차량은 사전 등록이 필요하며,
등록된 차량에 한해 지정된 시간 동안 주차할 수 있습니다.

[근거]

00아파트 주차관리규정
제15조 제2항
p.8

"....."

이를 통해 사용자가 AI의 답변을 직접 규정과 대조할 수 있도록 한다.

17. 검색 전략

기본 검색은 **Vector Similarity Search**를 사용한다.

사용자 질문
↓
Query Embedding
↓
OpenSearch
↓
유사도 높은 Chunk Top-K

다만 아파트 관리규정은 동일한 단어가 반복적으로 등장할 가능성이 높기 때문에 향후 필요하면 다음과 같이 확장할 수 있다.

Vector Search
+
Keyword Search
↓
Hybrid Search
↓
Reranking
↓
Top-K Context

이를 해커톤 MVP에서는 Vector Search + Metadata Filter 정도로 구현하는 것이 적절하다.

18. 문서 업데이트 Pipeline

관리자가 새로운 규정 PDF를 등록하거나 기존 규정을 업데이트하면 RAG Index도 함께 갱신한다.

```
관리자
↓
PDF 업로드
↓
S3
↓
Lambda
↓
Text Extraction
↓
Chunking
↓
Embedding
↓
OpenSearch Index 업데이트
```

기존 문서는 `version` 또는 활성 상태를 기준으로 관리하여 최신 규정이 검색되도록 한다.

19. 오류 및 예외 처리

19-1. 관련 규정 없음

검색 결과가 기준 유사도 이하인 경우 임의로 답변하지 않는다.

```
검색 결과 없음
↓
"관련 관리규정을 찾을 수 없습니다."
```

19-2. 질문 대상 아파트 불명확

예:

| "주차 규정 알려줘."

→ 로그인 사용자의 소속 아파트를 기본 대상으로 한다.

반면 여러 아파트를 언급했지만 대상이 명확하지 않은 경우에는 질문을 다시 확인하도록 한다.

19-3. 비공개 문서 검색 요청

사용자가 특정 아파트의 비공개 관리문서를 요청하더라도 검색 대상에서 제외한다.

```
B아파트
└─ is_public = false
    ↓
    검색 제외
```

19-4. 규정 외 질문

예:

| "오늘 서울 날씨 알려줘."

관리규정 RAG의 검색 대상이 아니므로 관리규정 기반 답변을 생성하지 않는다.

20. 보안 설계

20-1. 일반 서비스

JWT를 이용하여 사용자를 식별하고, Spring Boot에서 사용자의 `apartment_id` 를 기준으로 데이터 접근을 제한한다.

20-2. RAG

RAG에서는 검색 조건에 다음 Metadata Filter를 적용한다.

```
apartment_id
is_public
document_status
version
```

특히 `is_public = true` 조건을 적용하여 다른 아파트의 비공개 규정이 검색되는 것을 방지한다.

21. MVP 범위

이틀 동안 진행되는 해커톤이라는 점을 고려하여 RAG MVP는 다음 범위로 제한한다.

반드시 구현

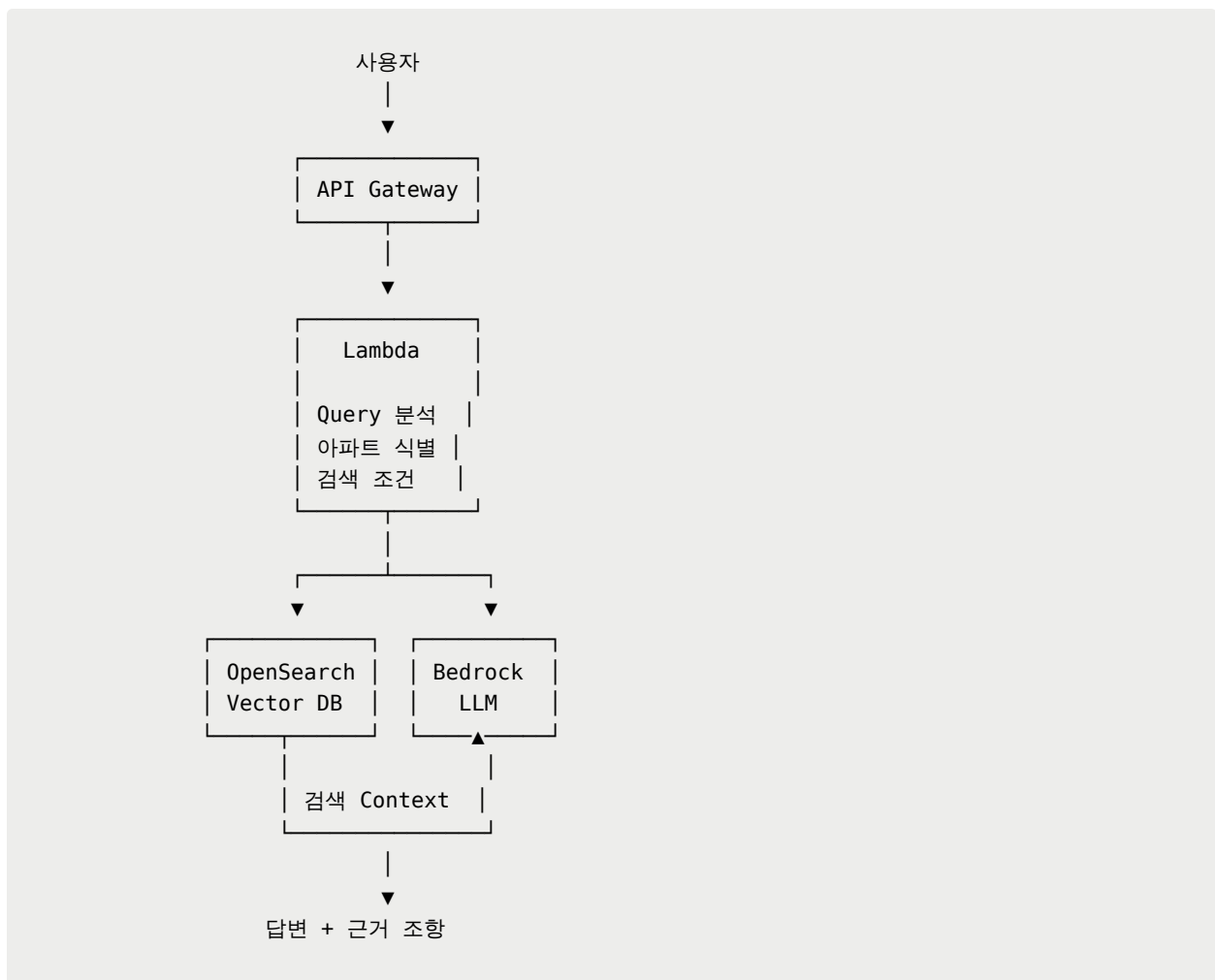
- ☒ 여러 아파트 규정 데이터 저장
- ☒ S3-PDF 저장
- ☒ PDF 텍스트 추출
- ☒ Chunking
- ☒ Bedrock Embedding
- ☒ OpenSearch Vector Search
- ☒ API Gateway
- ☒ Lambda
- ☒ 사용자 소속 아파트 기반 검색
- ☒ 특정 아파트 검색
- ☒ 공개 규정 Filter
- ☒ Bedrock 답변 생성
- ☒ 근거 조항 제공

후순위

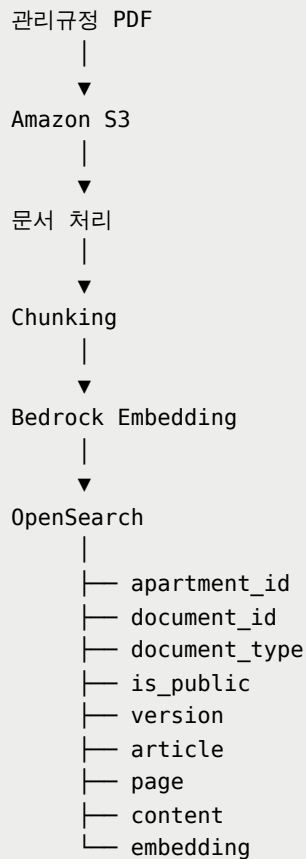
- ☐ Hybrid Search
- ☐ Reranker

- ☐ 대화 이력 기반 검색
- ☐ 규정 변경점 자동 비교
- ☐ 복잡한 Multi-Agent 구조
- ☐ 자동 문서 수집
- ☐ 고도화된 Query Rewrite

22. 최종 RAG 아키텍처



문서 처리 Pipeline



23. 핵심 설계 요약

이 RAG 시스템의 핵심은 단순히 "PDF를 넣고 질문하면 답변하는 챗봇"으로 만드는 것이 아니다.

핵심 구조는 다음과 같다.

다중 아파트 관리규정 → 아파트 Metadata 기반 검색 범위 제어 → Vector Search → Bedrock → 근거 조항 제공

특히 **apartment_id + is_public Metadata Filter**가 이 서비스에서 중요한 설계 요소다.

이를 통해 "내 아파트 규정은?", "다른 아파트 규정은?", "두 아파트 규정의 차이는?"이라는 세 가지 사용 시나리오를 하나의 RAG 구조에서 처리할 수 있다.